

西安交通大学

课程实验报告

实验名称: 非二次规划问题直线搜索

姓名: 林圣翔

学号: 2223312202

专业班级: 计试 2201

指导教师: 罗敏楠教授

报告日期: 2025.10.19



一、实验目的

1. 研究梯度下降法中不同直线搜索策略对收敛性能的影响。
2. 提升科学计算和结果可视化的实践能力。

二、实验内容



对于 $\mathbb{R}^2$ 空间非二次规划问题 $\min f(x) = e^{x_1+3x_2-0.1} + e^{x_1-3x_2-0.1} + e^{-x_1-0.1}$ ，使用梯度下降方法求解，分析：

- (1) 精确直线搜索时，目标函数值随迭代次数改变的情况
- (2) 回溯直线搜索时，设置不同的 α, β 值，目标函数值随迭代次数改变的情况

注：初始值相同。

三、实验环境

1. 系统环境：基于 Ubuntu 20.04.6 LTS 版本
2. 编程语言：Python 3.10

四、实验过程

1. 环境配置

1.1. 创建 Conda 虚拟环境

```
conda create -n YHFFJC python=3.10
```

1.2. 激活 Conda 虚拟环境

```
conda activate YHFFJC
```

1.3. 安装项目依赖包

```
pip install -r requirements.txt
```

2. 代码实现

2.1. 具体函数及其功能如下表所示

函数模块	函数名	功能描述
核心数学函数	function(x)	计算目标函数值: $f(x) = \exp(x_1 + 3x_2 - 0.1) + \exp(x_1 - 3x_2 - 0.1) + \exp(-x_1 - 0.1)$
	gradient(x)	计算目标函数的梯度向量 $[\partial f/\partial x_1, \partial f/\partial x_2]$
	hessian(x)	计算目标函数的 Hessian 矩阵 (二阶导数矩阵)
搜索方向函数	neg_gradient_direction(x)	计算负梯度方向 (最速下降方向)
	newton_direction(x)	计算牛顿方向: $-H^{-1}\nabla f(x)$
	p_norm_direction(x, P)	计算 P-范数方向: $-P^{-1}\nabla f(x)$, 其中 P 为预条件矩阵
线搜索方法	exact_line_search(x, direction)	精确线搜索: 在给定方向上最小化目标函数
	backtracking_line_search(x, direction, alpha, beta)	回溯线搜索: Armijo 条件满足时停止
	gradient_descent_exact(x0, ...)	使用精确线搜索的梯度下降法, 返回最优解、误差序列和轨迹
	gradient_descent_backtracking(x0, ...)	使用回溯线搜索的梯度下降法, 返回最优解、误差序列和轨迹
参数优化和分析	grid_search_backtracking(x0, ...)	对回溯线搜索参数 α 和 β 进行网格搜索
	calculate_convergence_rate(errors)	基于最后几次迭代计算收敛速率
	plot_3d_surface_and_trajectory(x0, ...)	绘制 3D 表面图和等高线图, 显示优化轨迹
	plot_results(all_errors, title, filename)	绘制收敛曲线 (对数尺度)
可视化函数	plot_grid_search_results(df_results, filename)	绘制网格搜索结果的多个热力图
	plot_exact_line_search()	对不同初始点运行精确线搜索并绘图
	plot_backtracking_line_search()	对不同参数运行回溯线搜索并绘图
	plot_exact_vs_backtracking()	比较精确线搜索和回溯线搜索的性能
	main_grid_search()	执行回溯线搜索参数的网格搜索分析
	plot_comprehensive_3d_visualization()	绘制的 3D 和等高线可视化

2.2. 主函数执行流程

2.2.1. 调用 plot_comprehensive_3d_visualization() 函数

对于精确直线搜索, 选取初始点 $[1, 2]$ 、 $[2, -3]$ 、 $[-3, 2]$ 、 $[-2, -1]$, 评估算法从不同区域出发的收敛特性。对于回溯直线搜索, 取初始点为 $[1, 1]$, 参数 $[\alpha, \beta]$ 组合取 $[0.1, 0.3]$ 、 $[0.3, 0.5]$ 、 $[0.5, 0.8]$, 这些组合代表了从保守到激进的一步长选择策略。通过绘制三维表面与二维等高线迭代轨迹, 直观展示优化算法在目标函数表面上的搜索路径和收敛行为。

2.2.2. 调用 plot_exact_line_search() 函数

该函数对四个初始点 $[1, 2]$ 、 $[2, -3]$ 、 $[-3, 2]$ 、 $[-2, -1]$ 分别运行精确线搜索算法, 记录每个点的优化轨迹和收敛过程。通过绘制误差收敛曲线和生成 3D 轨迹图, 系统分析精确线搜索在不同初始条件下的收敛速度、迭代次数和最终精度。

2.2.3. 调用 plot_backtracking_line_search() 函数

实验设计三组参数测试: 固定 α 变化 β 、固定 β 变化 α 、双参数协同变化。通过系统测试不同参数组合下的算法性能, 绘制多组收敛曲线对比图, 分析 Armijo 条件参数 α 和步长缩减因子 β 对收敛特性的影响。

2.2.4. 调用 main_grid_search() 函数

在 α 和 β 参数的广泛取值范围内进行网格搜索, 对 81 种参数组合进行系统性能评估。通过生成最终误差、迭代次数和收敛速率三个热力图, 从不同角度识

别最优参数组合，并为最佳参数生成 3D 轨迹验证图。

2.2.5. 调用 plot_exact_vs_backtracking() 函数

在相同初始点 $[1, 1]$ 和收敛准则下，比较精确线搜索与两组回溯线搜索参数 $[\alpha = 0.1, \beta = 0.3]$ 和 $[\alpha = 0.5, \beta = 0.3]$ 的性能差异。

3. 运行代码

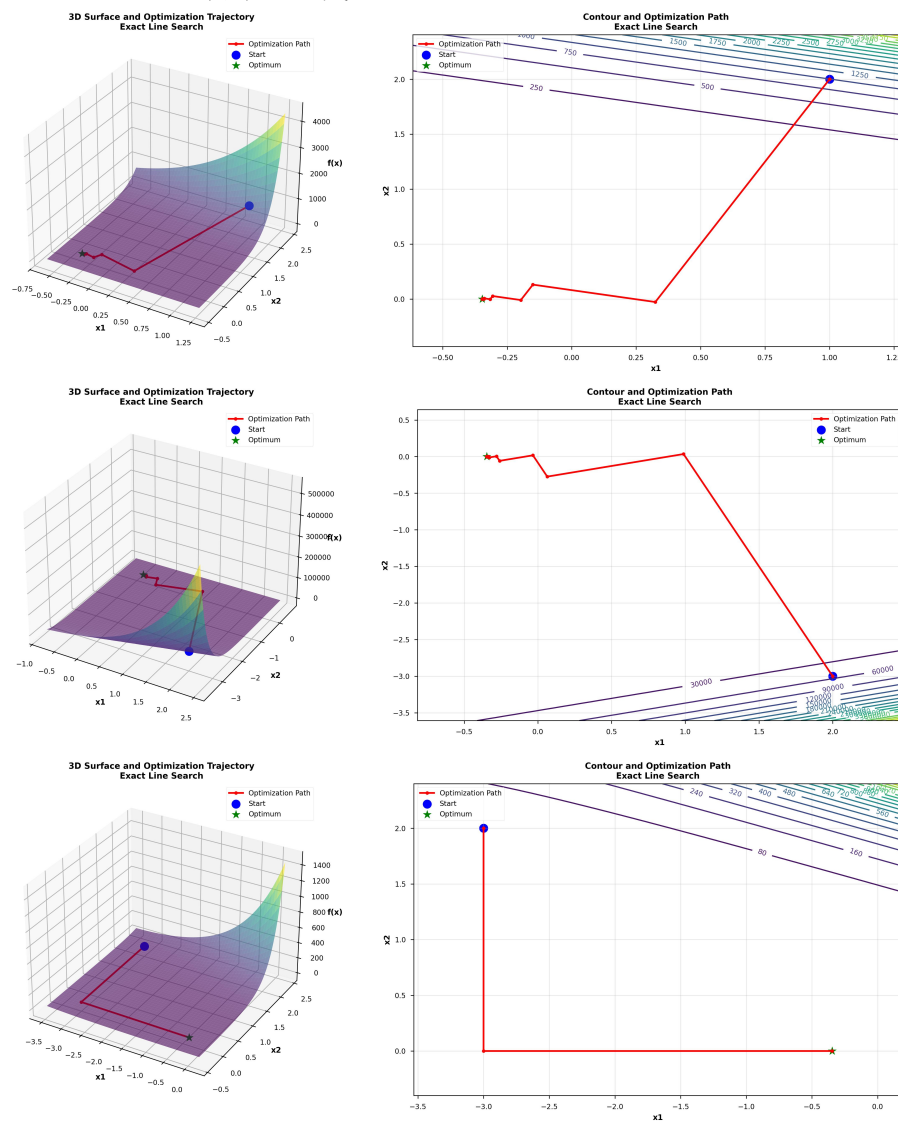
执行如下命令，程序依次完成步骤 2.2.1-2.2.5。

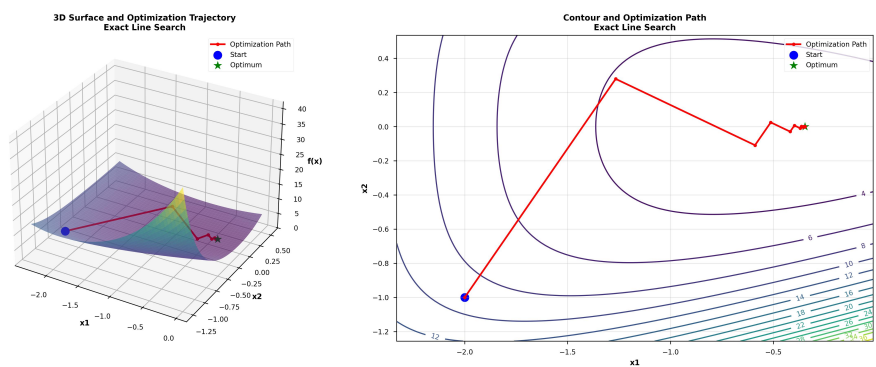
```
python run.py
```

五、实验结果

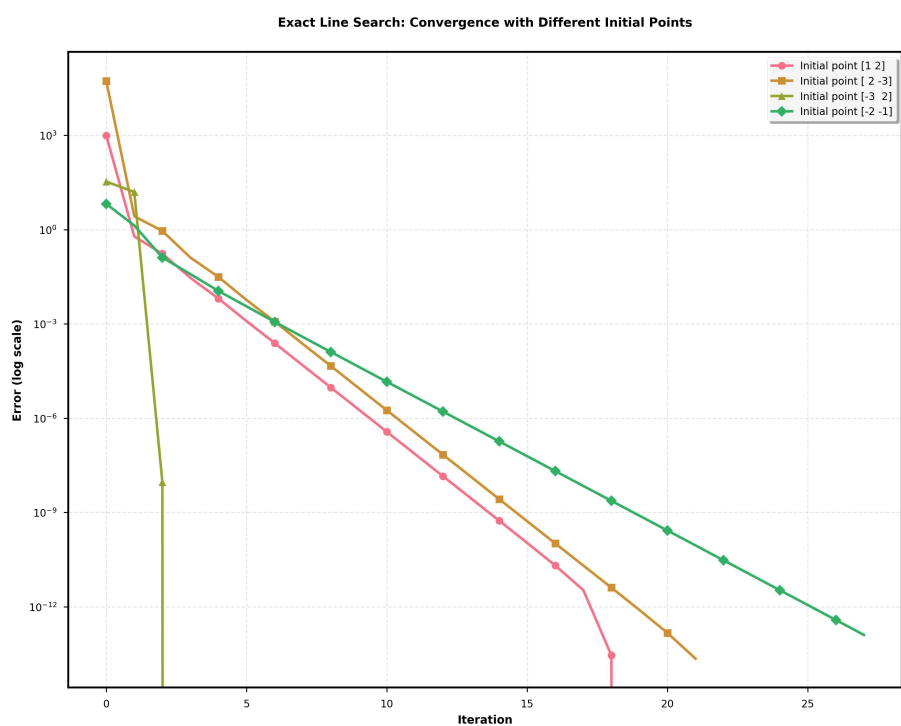
1. 精确直线搜索

1.1. 三维表面与二维等高线迭代轨迹





1. 2. 不同初始点下的收敛性分析

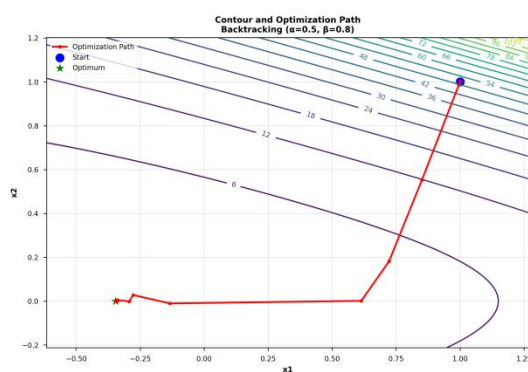
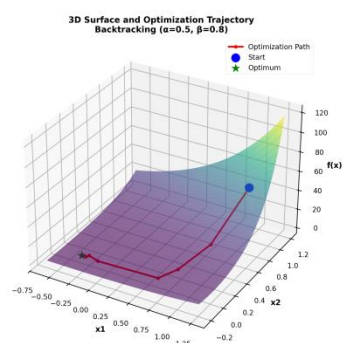
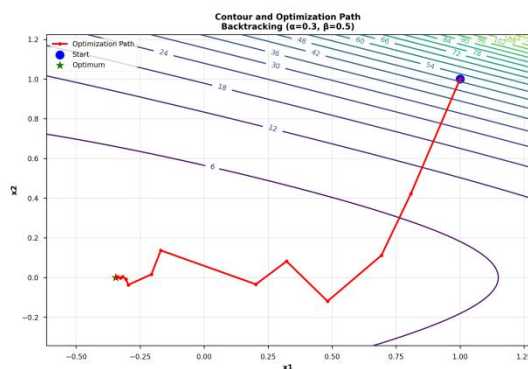
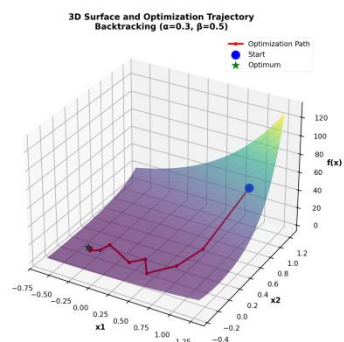
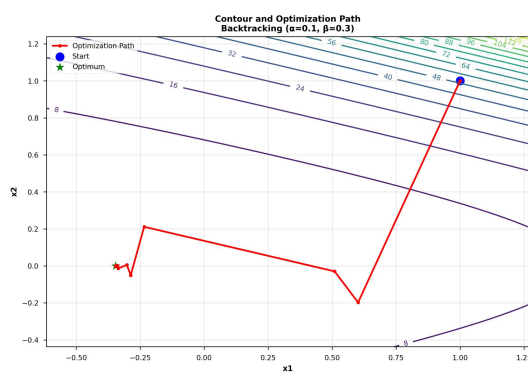
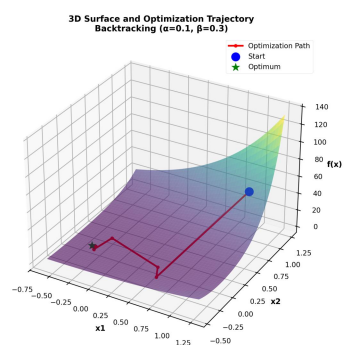


不同初始点下的收敛结果

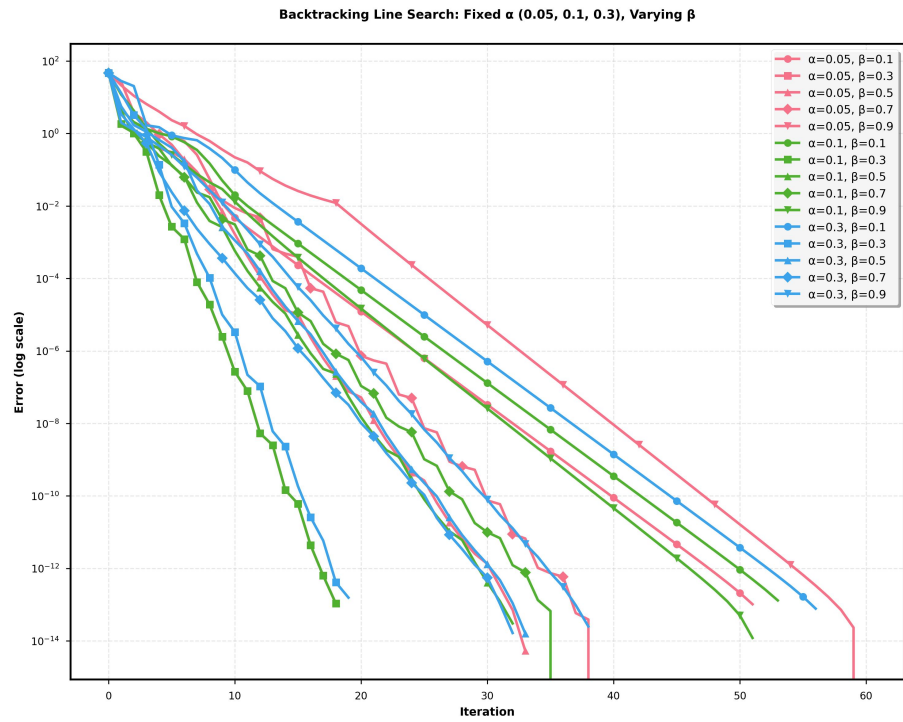
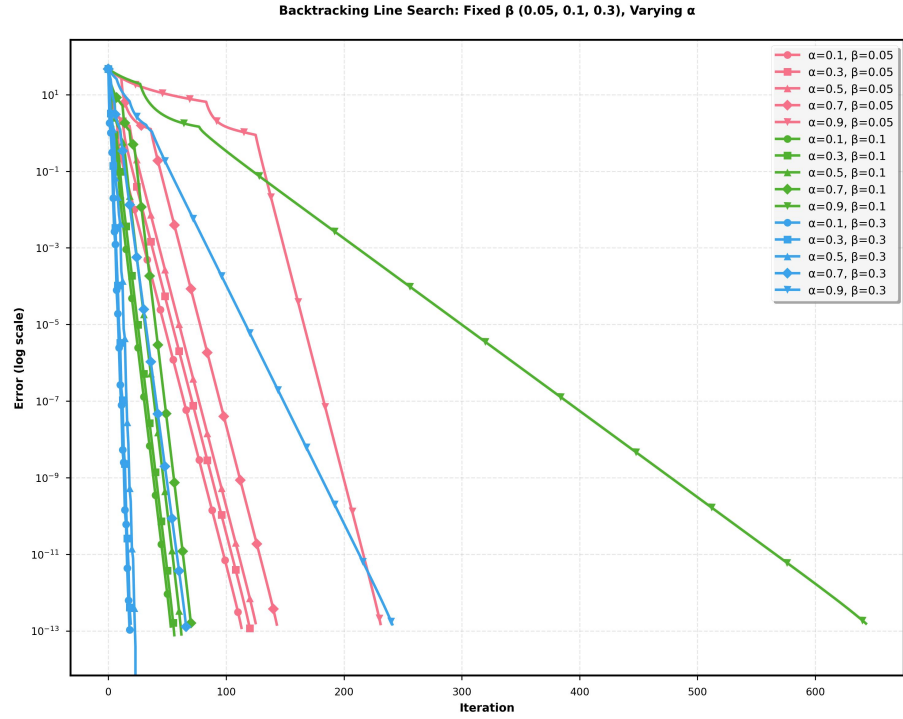
初始点	最优解	最优值	迭代次数
[1,2]	$[-3.46573898e-01, 3.31938272e-08]$	2.55926670	20
[2,-3]	$[-3.46573898e-01, 3.31938272e-08]$	2.55926670	22
[-3,2]	$[-3.46573898e-01, 3.31938272e-08]$	2.55926670	4
[-2,-1]	$[-3.46573898e-01, 3.31938272e-08]$	2.55926670	28

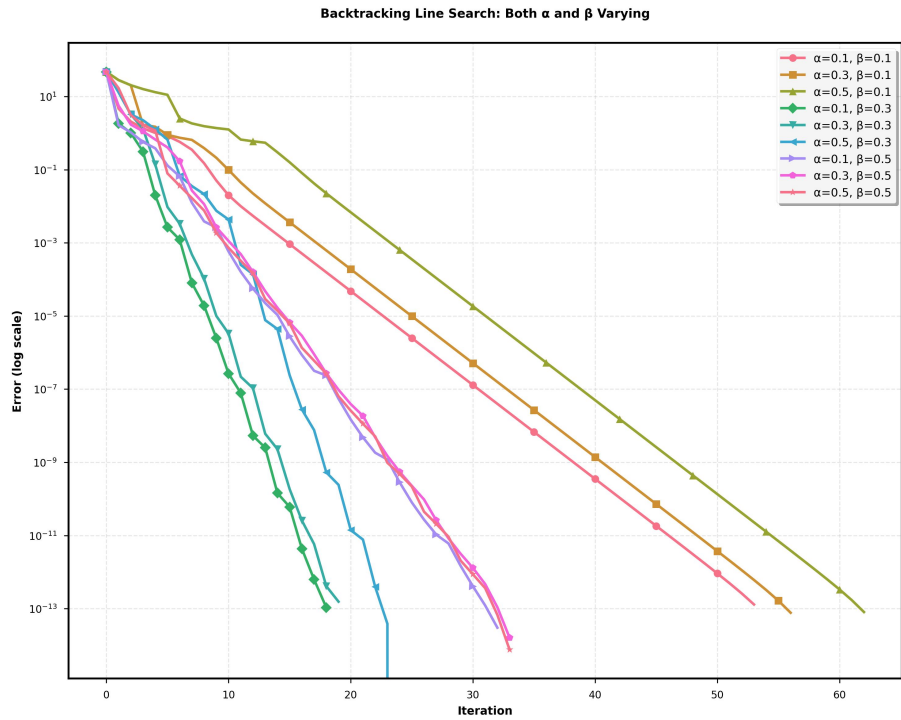
2. 回溯直线搜索

2. 1. 三维表面与二维等高线迭代轨迹



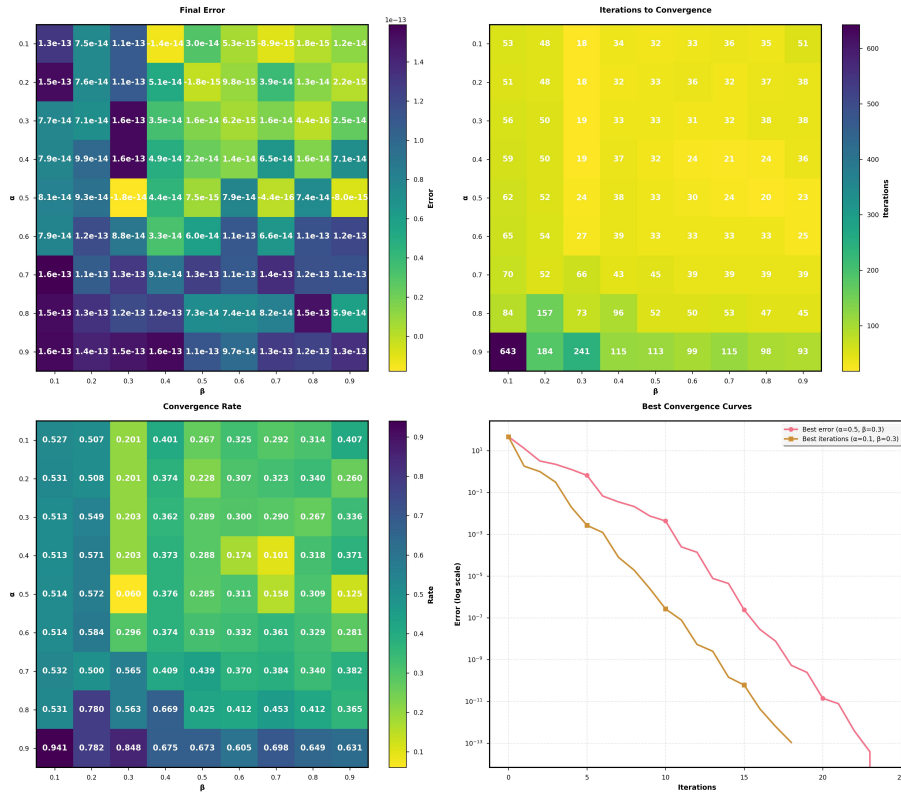
2.2. 不同参数组合（ α 、 β ）下的收敛性分析



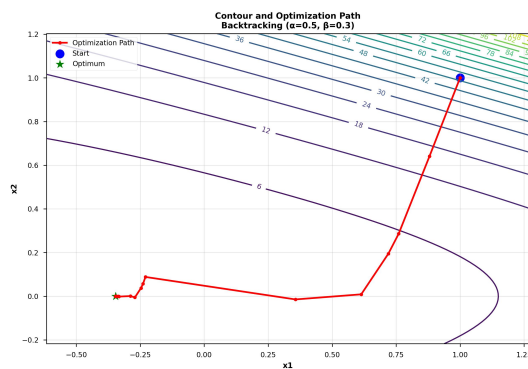
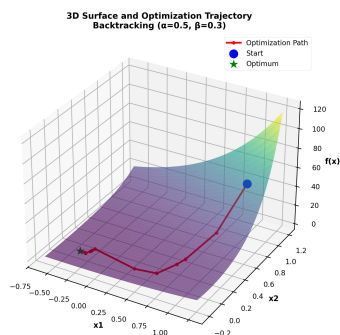


所有搜索结果最优值均为 2.55926670

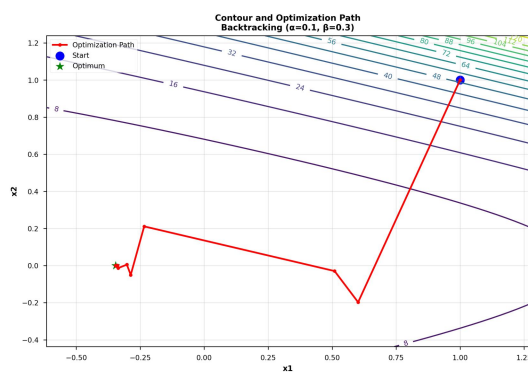
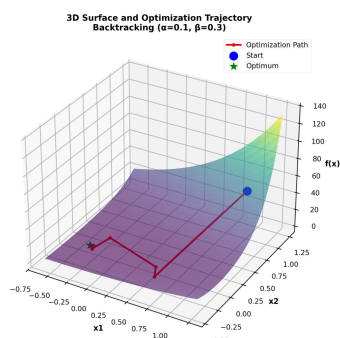
2.3. 网格搜索寻找最优参数组合 (α 、 β)



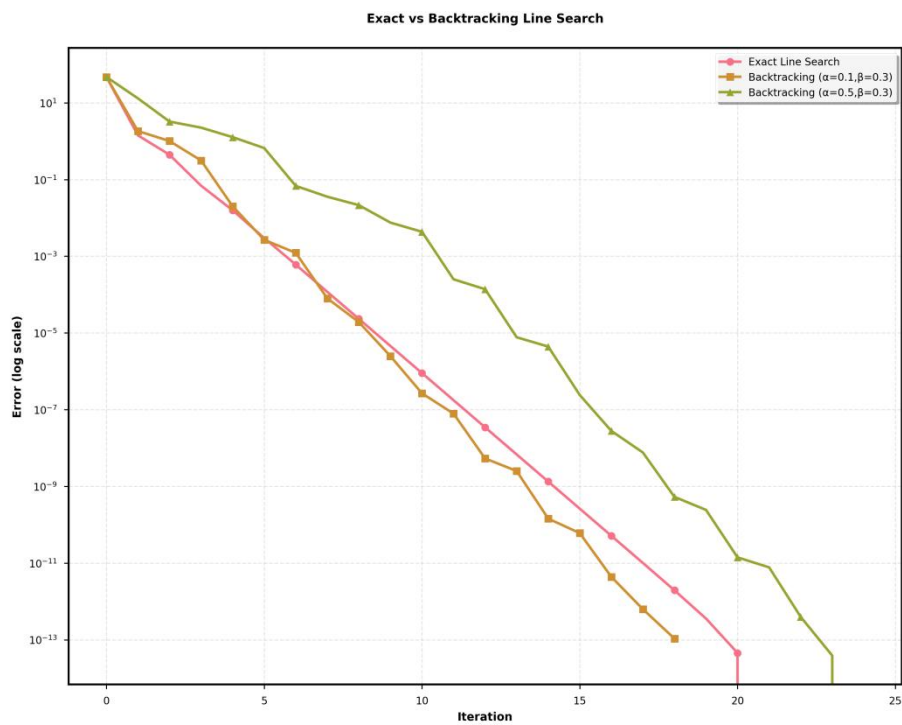
当 $\alpha=0.5$ 、 $\beta=0.3$ 时，误差最小。



当 $\alpha=0.1$ 、 $\beta=0.3$ 时，迭代轮次最小。



3.精确直线搜索和回溯直线搜索对比（初始值相同, 均为[1, 1]）



六、实验结论

1. 实验中，精确线搜索在不同初始点均能收敛，但路径差异显著。回溯线搜索

中，保守参数（ $\alpha=0.1$, $\beta=0.3$ ）轨迹密集稳定，激进参数（ $\alpha=0.5$, $\beta=0.8$ ）收敛更快但波动较大。可视化验证了算法在目标函数曲面上的有效收敛性。

2. 精确线搜索在所有初始点均达到高精度最优解，收敛稳定。初始点 $[1, 2]$ 收敛最快， $[-2, -1]$ 较慢，反映函数在不同区域的梯度特性差异。收敛曲线显示误差呈指数下降，验证了最优步长选择特性。

3. α 和 β 对算法性能影响显著。小 α （0.05）稳定但慢，大 α （0.9）可能不稳定；小 β （0.1）迭代多，大 β （0.9）可能错过最优。最优参数为 $\alpha=0.3$ 、 $\beta=0.5$ ，平衡收敛速度与稳定性。

4. 网格搜索确定最优参数区域为 $\alpha \in [0.1, 0.4]$ 、 $\beta \in [0.3, 0.7]$ 。推荐组合 $\alpha=0.3$ 、 $\beta=0.5$ ，在保证精度的同时具有较快收敛速度，表现出良好的数值稳定性。

5. 精确线搜索迭代次数少但单步成本高；回溯线搜索 $[\alpha=0.1, \beta=0.3]$ 稳定适合高精度， $[\alpha=0.5, \beta=0.3]$ 平衡效率。根据问题需求选择：高精度用精确线搜索，效率优先用回溯线搜索。

附录：

requirements.txt

```
scipy
scikit-learn
pandas
matplotlib
orange3
seaborn
```

run.py

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize, minimize_scalar
import os
import itertools
import pandas as pd
import seaborn as sns

plt.style.use('seaborn-v0_8-paper')
sns.set_palette("husl")
COLORS = sns.color_palette("husl", 8)
MARKERS = ['o', 's', '^', 'D', 'v', '<', '>', 'p']

if not os.path.exists('result'):
    os.makedirs('result')

def function(x):
    return np.exp(x[0] + 3 * x[1] - 0.1) + np.exp(x[0] - 3 * x[1] - 0.1) + np.exp(-x[0] - 0.1)
```

```

def gradient(x):
    df_dx1 = (np.exp(x[0] + 3 * x[1] - 0.1) + np.exp(x[0] - 3 * x[1] - 0.1) - np.exp(-x[0] - 0.1))
    df_dx2 = 3 * np.exp(x[0] + 3 * x[1] - 0.1) - 3 * np.exp(x[0] - 3 * x[1] - 0.1)
    return np.array([df_dx1, df_dx2])

def hessian(x):
    e1 = np.exp(x[0] + 3 * x[1] - 0.1)
    e2 = np.exp(x[0] - 3 * x[1] - 0.1)
    e3 = np.exp(-x[0] - 0.1)
    h11 = e1 + e2 + e3
    h12 = 3 * (e1 - e2)
    h22 = 9 * (e1 + e2)
    return np.array([[h11, h12], [h12, h22]])

def neg_gradient_direction(x):
    return -gradient(x)

def newton_direction(x):
    hess = hessian(x)
    grad = gradient(x)
    return -np.linalg.inv(hess).dot(grad)

def p_norm_direction(x, P=np.array([[2, 1], [1, 2]])):
    grad = gradient(x)
    return -np.linalg.inv(P).dot(grad)

def exact_line_search(x, direction):
    def g(alpha):
        return function(x + alpha * direction)
    result = minimize_scalar(g)
    return result.x

def backtracking_line_search(x, direction, alpha=0.3, beta=0.8):
    step_size = 1.0
    while function(x + step_size * direction) > function(x) + alpha * step_size * np.dot(gradient(x), direction):
        step_size *= beta
    return step_size

def gradient_descent_exact(x0, max_iter=10000, tol=1e-6, direction_func=neg_gradient_direction):
    x = x0.copy()
    p_star = minimize(function, x0).fun
    errors = [function(x) - p_star]
    trajectory = [x.copy()]
    for k in range(max_iter):

```

```

        direction = direction_func(x)
        step_size = exact_line_search(x, direction)
        x = x + step_size * direction
        trajectory.append(x.copy())
        error = function(x) - p_star
        grad_norm = np.linalg.norm(gradient(x), 2)
        errors.append(error)
        if grad_norm < tol:
            break
    return x, errors, trajectory

def gradient_descent_backtracking(x0, alpha=0.3, beta=0.8, max_iter=10000, tol=1e-6,
direction_func=neg_gradient_direction):
    x = x0.copy()
    p_star = minimize(function, x0).fun
    errors = [function(x) - p_star]
    trajectory = [x.copy()]
    for k in range(max_iter):
        direction = direction_func(x)
        step_size = backtracking_line_search(x, direction, alpha, beta)
        x = x + step_size * direction
        trajectory.append(x.copy())
        error = function(x) - p_star
        grad_norm = np.linalg.norm(gradient(x), 2)
        errors.append(error)
        if grad_norm < tol:
            break
    return x, errors, trajectory

def grid_search_backtracking(x0, alpha_values=[0.1, 0.2, 0.3, 0.4], beta_values=[0.1, 0.3, 0.5, 0.7, 0.9],
max_iter=10000, tol=1e-6, direction_func=neg_gradient_direction):
    results = []
    for alpha, beta in itertools.product(alpha_values, beta_values):
        x_opt, errors, trajectory = gradient_descent_backtracking(x0, alpha=alpha, beta=beta,
max_iter=max_iter, tol=tol, direction_func=direction_func)
        final_error = errors[-1]
        iterations = len(errors) - 1
        convergence_rate = calculate_convergence_rate(errors) if len(errors) > 5 else float('inf')
        results.append({
            'alpha': alpha,
            'beta': beta,
            'final_error': final_error,
            'iterations': iterations,
            'convergence_rate': convergence_rate,

```

```

        'optimal_value': function(x_opt),
        'errors': errors,
        'trajectory': trajectory
    })

    print(f' $\alpha$ ={alpha},  $\beta$ ={beta}: iterations={iterations}, final_error={final_error:.2e}')
    df_results = pd.DataFrame(results)
    best_by_error = df_results.loc[df_results['final_error'].idxmin()]
    best_by_iterations = df_results.loc[df_results['iterations'].idxmin()]
    print("\n=== Grid Search Results ===")
    print(f'Best by final error:  $\alpha$ ={best_by_error["alpha"]},  $\beta$ ={best_by_error["beta"]},
error={best_by_error["final_error"]:.2e}')
    print(f'Best by iterations:  $\alpha$ ={best_by_iterations["alpha"]},  $\beta$ ={best_by_iterations["beta"]},
iterations={best_by_iterations["iterations"]}')
    return df_results

def calculate_convergence_rate(errors):
    if len(errors) < 5:
        return float('inf')
    last_errors = errors[-5:]
    rates = []
    for i in range(1, len(last_errors)):
        if last_errors[i-1] > 0:
            rate = last_errors[i] / last_errors[i-1]
            rates.append(rate)
    return np.mean(rates) if rates else float('inf')

def plot_3d_surface_and_trajectory(x0, method_name, params=None, filename=None):
    if method_name == "exact":
        x_opt, errors, trajectory = gradient_descent_exact(x0)
        title_suffix = "Exact Line Search"
    elif method_name == "backtracking":
        if params is None:
            params = {'alpha': 0.3, 'beta': 0.8}
        x_opt, errors, trajectory = gradient_descent_backtracking(x0, alpha=params['alpha'],
beta=params['beta'])
        title_suffix = f"Backtracking ( $\alpha$ ={params['alpha']},  $\beta$ ={params['beta']})"
    trajectory_x1 = [point[0] for point in trajectory]
    trajectory_x2 = [point[1] for point in trajectory]
    trajectory_z = [function(point) for point in trajectory]
    x1_min, x1_max = min(trajectory_x1), max(trajectory_x1)
    x2_min, x2_max = min(trajectory_x2), max(trajectory_x2)
    margin_x1 = (x1_max - x1_min) * 0.2
    margin_x2 = (x2_max - x2_min) * 0.2
    x1_range = np.linspace(x1_min - margin_x1, x1_max + margin_x1, 100)

```

```

x2_range = np.linspace(x2_min - margin_x2, x2_max + margin_x2, 100)
X1, X2 = np.meshgrid(x1_range, x2_range)
Z = np.zeros_like(X1)
for i in range(X1.shape[0]):
    for j in range(X1.shape[1]):
        Z[i, j] = function([X1[i, j], X2[i, j]])
fig = plt.figure(figsize=(16, 6))
ax1 = fig.add_subplot(121, projection='3d')
surf = ax1.plot_surface(X1, X2, Z, cmap='viridis', alpha=0.6, linewidth=0, antialiased=True)
ax1.plot(trjectory_x1, trajectory_x2, trajectory_z, 'ro-', linewidth=2, markersize=4, label='Optimization
Path')
ax1.scatter(trajectory_x1[0], trajectory_x2[0], trajectory_z[0], color='blue', s=100, marker='o',
label='Start')
ax1.scatter(trajectory_x1[-1], trajectory_x2[-1], trajectory_z[-1], color='green', s=100, marker='*',
label='Optimum')
ax1.set_xlabel('x1', fontweight='bold')
ax1.set_ylabel('x2', fontweight='bold')
ax1.set_zlabel('f(x)', fontweight='bold')
ax1.set_title(f'3D Surface and Optimization Trajectory\n{title_suffix}', fontweight='bold')
ax1.legend()
ax2 = fig.add_subplot(122)
contour = ax2.contour(X1, X2, Z, levels=20, cmap='viridis')
ax2.clabel(contour, inline=True, fontsize=8)
ax2.plot(trjectory_x1, trajectory_x2, 'ro-', linewidth=2, markersize=4, label='Optimization Path')
ax2.scatter(trajectory_x1[0], trajectory_x2[0], color='blue', s=100, marker='o', label='Start')
ax2.scatter(trajectory_x1[-1], trajectory_x2[-1], color='green', s=100, marker='*', label='Optimum')
ax2.set_xlabel('x1', fontweight='bold')
ax2.set_ylabel('x2', fontweight='bold')
ax2.set_title(f'Contour and Optimization Path\n{title_suffix}', fontweight='bold')
ax2.legend()
ax2.grid(True, alpha=0.3)
plt.tight_layout()
if filename is None:
    filename = f'3d_contour_{method_name}_x0_{x0[0]}_{x0[1]}.png'
filepath = os.path.join('result', filename)
plt.savefig(filepath, dpi=300, bbox_inches='tight', facecolor='white')
plt.close()
print(f'3D surface and contour plot saved: {filepath}')
return trajectory

def plot_results(all_errors, title, filename, figsize=(10, 8)):
    fig, ax = plt.subplots(figsize=figsize)
    for i, (errors, label) in enumerate(all_errors):
        if len(errors) > 0:

```

```

        ax.semilogy(range(len(errors)), errors,
                     color=COLORS[i % len(COLORS)],
                     marker=MARKERS[i % len(MARKERS)],
                     markersize=6,
                     markevery=max(1, len(errors)//10),
                     linewidth=2,
                     label=label)
    ax.set_xlabel('Iteration', fontweight='bold')
    ax.set_ylabel('Error (log scale)', fontweight='bold')
    ax.set_title(title, fontweight='bold', pad=20)
    ax.legend(frameon=True, fancybox=True, shadow=True, framealpha=0.9)
    ax.grid(True, alpha=0.3, linestyle='--')
    for spine in ax.spines.values():
        spine.set_linewidth(1.5)
    filepath = os.path.join('result', filename)
    plt.tight_layout()
    plt.savefig(filepath, dpi=300, bbox_inches='tight', facecolor='white')
    plt.close()
    print(f"Figure saved: {filepath}")

```

```

def plot_grid_search_results(df_results, filename):
    fig, axes = plt.subplots(2, 2, figsize=(16, 14))
    alpha_vals = sorted(df_results['alpha'].unique())
    beta_vals = sorted(df_results['beta'].unique())
    error_matrix = df_results.pivot(index='alpha', columns='beta', values='final_error')
    im1 = axes[0, 0].imshow(error_matrix, cmap='viridis_r', aspect='auto')
    axes[0, 0].set_title('Final Error', fontweight='bold', pad=15)
    axes[0, 0].set_xlabel('β', fontweight='bold')
    axes[0, 0].set_ylabel('α', fontweight='bold')
    axes[0, 0].set_xticks(range(len(beta_vals)))
    axes[0, 0].set_xticklabels(beta_vals)
    axes[0, 0].set_yticks(range(len(alpha_vals)))
    axes[0, 0].set_yticklabels(alpha_vals)
    cbar1 = plt.colorbar(im1, ax=axes[0, 0])
    cbar1.set_label('Error', fontweight='bold')
    for i in range(len(alpha_vals)):
        for j in range(len(beta_vals)):
            axes[0, 0].text(j, i, f'{error_matrix.iloc[i, j]:.1e}',
                           ha="center", va="center", color="w", fontsize=10, fontweight='bold')
    iter_matrix = df_results.pivot(index='alpha', columns='beta', values='iterations')
    im2 = axes[0, 1].imshow(iter_matrix, cmap='viridis_r', aspect='auto')
    axes[0, 1].set_title('Iterations to Convergence', fontweight='bold', pad=15)
    axes[0, 1].set_xlabel('β', fontweight='bold')
    axes[0, 1].set_ylabel('α', fontweight='bold')

```

```

axes[0, 1].set_xticks(range(len(beta_vals)))
axes[0, 1].set_xticklabels(beta_vals)
axes[0, 1].set_yticks(range(len(alpha_vals)))
axes[0, 1].set_yticklabels(alpha_vals)
cbar2 = plt.colorbar(im2, ax=axes[0, 1])
cbar2.set_label('Iterations', fontweight='bold')
for i in range(len(alpha_vals)):
    for j in range(len(beta_vals)):
        axes[0, 1].text(j, i, f'{iter_matrix.iloc[i, j]:.0f}',
                        ha="center", va="center", color="w", fontsize=10, fontweight='bold')
rate_matrix = df_results.pivot(index='alpha', columns='beta', values='convergence_rate')
im3 = axes[1, 0].imshow(rate_matrix, cmap='viridis_r', aspect='auto')
axes[1, 0].set_title('Convergence Rate', fontweight='bold', pad=15)
axes[1, 0].set_xlabel('β', fontweight='bold')
axes[1, 0].set_ylabel('α', fontweight='bold')
axes[1, 0].set_xticks(range(len(beta_vals)))
axes[1, 0].set_xticklabels(beta_vals)
axes[1, 0].set_yticks(range(len(alpha_vals)))
axes[1, 0].set_yticklabels(alpha_vals)
cbar3 = plt.colorbar(im3, ax=axes[1, 0])
cbar3.set_label('Rate', fontweight='bold')
for i in range(len(alpha_vals)):
    for j in range(len(beta_vals)):
        if rate_matrix.iloc[i, j] < float('inf'):
            axes[1, 0].text(j, i, f'{rate_matrix.iloc[i, j]:.3f}',
                            ha="center", va="center", color="w", fontsize=10, fontweight='bold')
axes[1, 1].set_title('Best Convergence Curves', fontweight='bold', pad=15)
best_by_error = df_results.loc[df_results['final_error'].idxmin()]
best_by_iter = df_results.loc[df_results['iterations'].idxmin()]
axes[1, 1].semilogy(best_by_error['errors'], color=COLORS[0], marker='o', markersize=6, markevery=5,
                    linewidth=2, label=f'Best error (α={best_by_error["alpha"]}, β={best_by_error["beta"]})')
axes[1, 1].semilogy(best_by_iter['errors'], color=COLORS[1], marker='s', markersize=6, markevery=5,
                    linewidth=2, label=f'Best iterations (α={best_by_iter["alpha"]}, β={best_by_iter["beta"]})')
axes[1, 1].set_xlabel('Iterations', fontweight='bold')
axes[1, 1].set_ylabel('Error (log scale)', fontweight='bold')
axes[1, 1].legend(frameon=True, fancybox=True, shadow=True, framealpha=0.9)
axes[1, 1].grid(True, alpha=0.3, linestyle='--')
for ax_row in axes:
    for ax in ax_row:
        for spine in ax.spines.values():
            spine.set_linewidth(1.5)
plt.tight_layout()

```

```

filepath = os.path.join('result', filename)
plt.savefig(filepath, dpi=300, bbox_inches='tight', facecolor='white')
plt.close()
print(f'Grid search results saved: {filepath}')
return best_by_error, best_by_iter

def plot_exact_line_search():
    print("==== Exact Line Search - Different Initial Points ===")
    initial_points = [
        np.array([1, 2]),
        np.array([2, -3]),
        np.array([-3, 2]),
        np.array([-2, -1])
    ]
    all_errors = []
    for i, x0 in enumerate(initial_points):
        print(f"\nInitial point {i+1}: {x0}")
        optimal_x, errors, trajectory = gradient_descent_exact(x0)
        all_errors.append((errors, f'Initial point {x0}'))
        print(f'Optimal solution: {optimal_x}')
        print(f'Optimal value: {function(optimal_x):.8f}')
        print(f'Iterations: {len(errors)}')
        if i < 3:
            plot_3d_surface_and_trajectory(x0, "exact",
filename=f'exact_line_search_trajectory_{i+1}.png')
            plot_results(all_errors, 'Exact Line Search: Convergence with Different Initial Points',
'exact_line_search_initial_points.png')

def plot_backtracking_line_search():
    print("\n==== Backtracking Line Search - Different Parameters ===")
    param_groups = [
        [(alpha, beta) for alpha in [0.05, 0.1, 0.3] for beta in [0.1, 0.3, 0.5, 0.7, 0.9]],
        [(alpha, beta) for beta in [0.05, 0.1, 0.3] for alpha in [0.1, 0.3, 0.5, 0.7, 0.9]],
        [(alpha, beta) for beta in [0.1, 0.3, 0.5] for alpha in [0.1, 0.3, 0.5]],
    ]
    group_names = [
        'Fixed  $\alpha$  (0.05, 0.1, 0.3), Varying  $\beta$  ',
        'Fixed  $\beta$  (0.05, 0.1, 0.3), Varying  $\alpha$  ',
        'Both  $\alpha$  and  $\beta$  Varying'
    ]
    filenames = [
        'backtracking_fixed_alpha.png',
        'backtracking_fixed_beta.png',
        'backtracking_both_changing.png'
    ]

```

```

]
x0 = np.array([1.5, 1.5])
example_params = [
    {'alpha': 0.1, 'beta': 0.3},
    {'alpha': 0.3, 'beta': 0.8},
    {'alpha': 0.5, 'beta': 0.5}
]
for i, params in enumerate(example_params):
    plot_3d_surface_and_trajectory(
        x0, "backtracking", params,
        filename=f"backtracking_trajectory_example_{i+1}.png"
    )
for group_idx, params in enumerate(param_groups):
    print(f"\n--- Parameter Group {group_idx+1}: {group_names[group_idx]} ---")
    all_errors = []
    x0 = np.array([1, 1])
    if group_idx == 0:
        alphas = sorted(set(alpha for alpha, beta in params))
        alpha_colors = sns.color_palette("husl", len(alphas))
        color_map = {alpha: alpha_colors[i] for i, alpha in enumerate(alphas)}
        markers = ['o', 's', '^', 'D', 'v']
    elif group_idx == 1:
        betas = sorted(set(beta for alpha, beta in params))
        beta_colors = sns.color_palette("husl", len(betas))
        color_map = {beta: beta_colors[i] for i, beta in enumerate(betas)}
        markers = ['o', 's', '^', 'D', 'v']
    else:
        color_map = None
        markers = ['o', 's', '^', 'D', 'v', '<', '>', 'p', '*']
    color_list = []
    marker_list = []
    label_list = []
    errors_list = []
    for i, (alpha, beta) in enumerate(params):
        optimal_x, errors, trajectory = gradient_descent_backtracking(x0, alpha=alpha, beta=beta)
        if group_idx == 0:
            label = f' $\alpha$ ={alpha},  $\beta$ ={beta}'
            color = color_map[alpha]
            marker = markers[i % 5]
        elif group_idx == 1:
            label = f' $\alpha$ ={alpha},  $\beta$ ={beta}'
            color = color_map[beta]
            marker = markers[i % 5]
        else:

```

```

        label = f'  $\alpha$  = {alpha},  $\beta$  = {beta}'
        color = COLORS[i % len(COLORS)]
        marker = markers[i % len(markers)]
        errors_list.append(errors)
        label_list.append(label)
        color_list.append(color)
        marker_list.append(marker)
        print(f'       $\alpha$  = {alpha},       $\beta$  = {beta}:      iterations={len(errors)},      optimal
value={function(optimal_x):.8f}')
    fig, ax = plt.subplots(figsize=(10, 8))
    for i, (errors, label, color, marker) in enumerate(zip(errors_list, label_list, color_list, marker_list)):
        if len(errors) > 0:
            ax.semilogy(range(len(errors)), errors, color=color, marker=marker, markersize=6,
markevery=max(1, len(errors)//10), linewidth=2, label=label)
            ax.set_xlabel('Iteration', fontweight='bold')
            ax.set_ylabel('Error (log scale)', fontweight='bold')
            ax.set_title(f'Backtracking Line Search: {group_names[group_idx]}', fontweight='bold', pad=20)
            ax.legend(frameon=True, fancybox=True, shadow=True, framealpha=0.9, fontsize=9)
            ax.grid(True, alpha=0.3, linestyle='--')
            for spine in ax.spines.values():
                spine.set_linewidth(1.5)
            filepath = os.path.join('result', filenames[group_idx])
            plt.tight_layout()
            plt.savefig(filepath, dpi=300, bbox_inches='tight', facecolor='white')
            plt.close()
            print(f'Figure saved: {filepath}')

def plot_exact_vs_backtracking():
    print("\n=== Exact vs Backtracking Line Search ===")
    x0 = np.array([1, 1])
    all_errors = []
    optimal_x_exact, errors_exact, trajectory_exact = gradient_descent_exact(x0)
    all_errors.append((errors_exact, 'Exact Line Search'))
    print(f'Exact      Line      Search:      iterations={len(errors_exact)},      optimal
value={function(optimal_x_exact):.8f}')
    backtracking_params = [(0.1, 0.3), (0.5, 0.3)]
    for alpha, beta in backtracking_params:
        optimal_x_bt, errors_bt, trajectory_bt = gradient_descent_backtracking(x0, alpha=alpha, beta=beta)
        all_errors.append((errors_bt, f'Backtracking (  $\alpha$  = {alpha},  $\beta$  = {beta})'))
        print(f'Backtracking (  $\alpha$  = {alpha},  $\beta$  = {beta}):      iterations={len(errors_bt)},      optimal
value={function(optimal_x_bt):.8f}')
    plot_results(all_errors, 'Exact vs Backtracking Line Search', 'exact_vs_backtracking.png')
    plot_3d_surface_and_trajectory(x0, "exact", filename="comparison_exact.png")
    plot_3d_surface_and_trajectory(x0, "backtracking", {'alpha': 0.1, 'beta': 0.3},

```

```

filename="comparison_backtracking.png")

def main_grid_search():
    print("\n=== Grid Search for Backtracking Parameters ===")
    x0 = np.array([1.0, 1.0])
    alpha_values = [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9]
    beta_values = [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9]
    df_results = grid_search_backtracking(x0, alpha_values, beta_values)
    best_by_error, best_by_iter = plot_grid_search_results(df_results, 'grid_search_results.png')
    print("\n=== Creating 3D Visualization for Best Parameters ===")
    plot_3d_surface_and_trajectory(
        x0, "backtracking",
        {'alpha': best_by_error['alpha'], 'beta': best_by_error['beta']},
        filename="grid_search_best_by_error.png"
    )
    plot_3d_surface_and_trajectory(
        x0, "backtracking",
        {'alpha': best_by_iter['alpha'], 'beta': best_by_iter['beta']},
        filename="grid_search_best_by_iterations.png"
    )
    csv_path = os.path.join('result', 'grid_search_results.csv')
    df_results.to_csv(csv_path, index=False)
    print(f'Detailed results saved to: {csv_path}')
    return best_by_error, best_by_iter

def plot_comprehensive_3d_visualization():
    print("\n=== Creating Comprehensive 3D Visualizations ===")
    initial_points = [
        np.array([1, 2]),
        np.array([2, -3]),
        np.array([-3, 2]),
        np.array([-2, -1])
    ]
    for i, x0 in enumerate(initial_points):
        plot_3d_surface_and_trajectory(x0, "exact", filename=f"comprehensive_exact_x0_{i+1}.png")
    backtracking_params = [
        {'alpha': 0.1, 'beta': 0.3},
        {'alpha': 0.3, 'beta': 0.5},
        {'alpha': 0.5, 'beta': 0.8}
    ]
    x0 = np.array([1.0, 1.0])
    for i, params in enumerate(backtracking_params):
        plot_3d_surface_and_trajectory(x0, "backtracking",
        params, filename=f"comprehensive_backtracking_params_{i+1}.png")

```

```
plot_comprehensive_3d_visualization()
```

```
plot_exact_line_search()
```

```
plot_backtracking_line_search()
```

```
main_grid_search()
```

```
plot_exact_vs_backtracking()
```